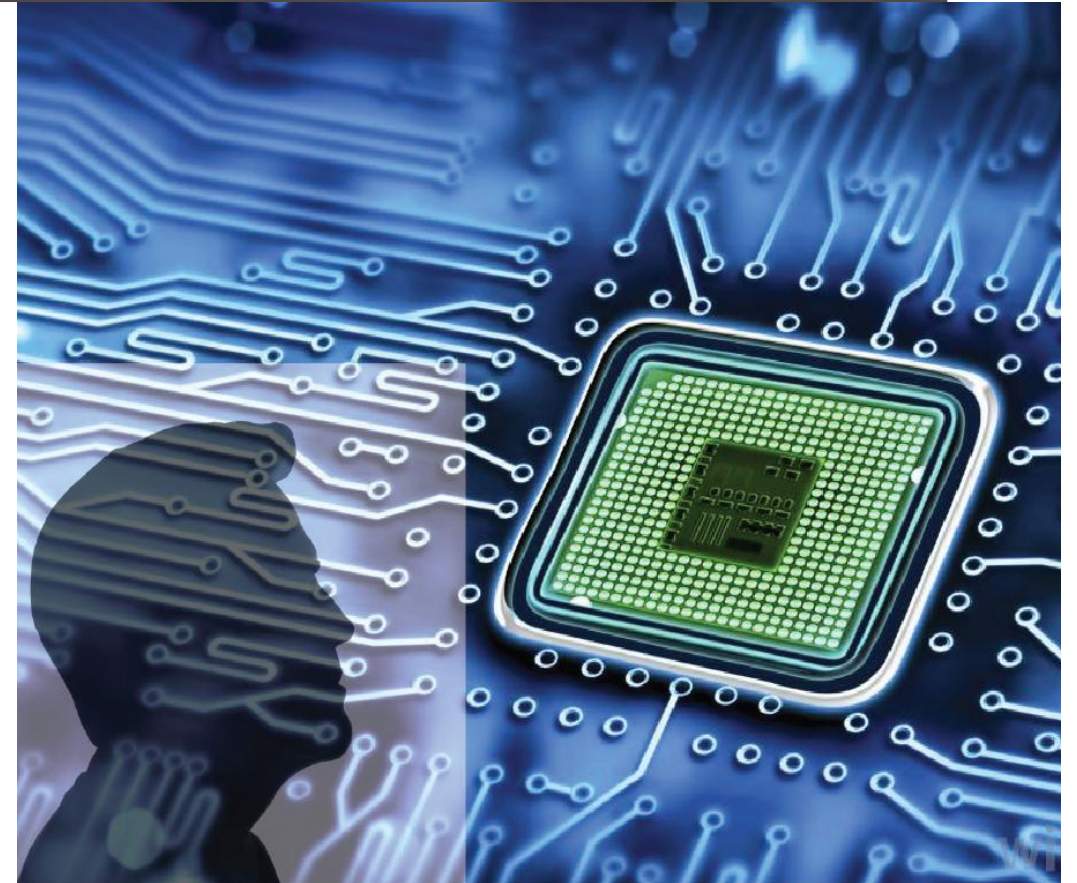


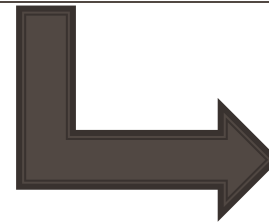
ADVANCED DIGITAL DESIGN AND VERIFICATION

Week-9 Lecture-1 SystemVerilog

Sneh Saurabh
14th October, 2025



SystemVerilog



Inheritance

Parametrized Classes

- Data type can be added as parameter to class
 - Similar to template in C++
- The data type is specified when the handle to that class is declared
 - Specialization

```
ncsim> run
8.000000
6.000000
4.000000
2.000000
0.000000
```

```
1 class Stack #(type T=int);
2   local T stack[];
3   local int top;
4
5   function new(int size=100);
6     stack = new[size];
7   endfunction
8
9   function void push(input T i);
10    stack[top++] = i;
11  endfunction
12
13  function T pop();
14    return stack[--top];
15  endfunction
16
17 endclass: Stack
18
19 module top;
20   initial begin
21
22     Stack #(real) rStack;
23     rStack = new(10);
24
25     for (int i=0; i<5; i++)
26       rStack.push(i*2.0);
27
28     for (int i=0; i<5; i++)
29       $display("%f", rStack.pop());
30   end
31 endmodule
```

Abstract Classes and Pure Virtual Methods

Abstract Class

- A class which cannot be instantiated (i.e., object cannot be created using new)
- **virtual** keyword added to the beginning of class declaration

Pure Virtual Method

- It contains only the prototype and no body
- Can be declared using the keyword **pure** along with **virtual** keyword
- A pure virtual method can only be declared in an abstract class

To create objects with abstract class

- Extend the class
- Provide implementation of all the pure virtual methods

```
virtual class BaseTr;  
    static int count;  
    int id;  
  
    function new();  
        id = count++;  
    endfunction  
  
    pure virtual function void display();  
endclass
```

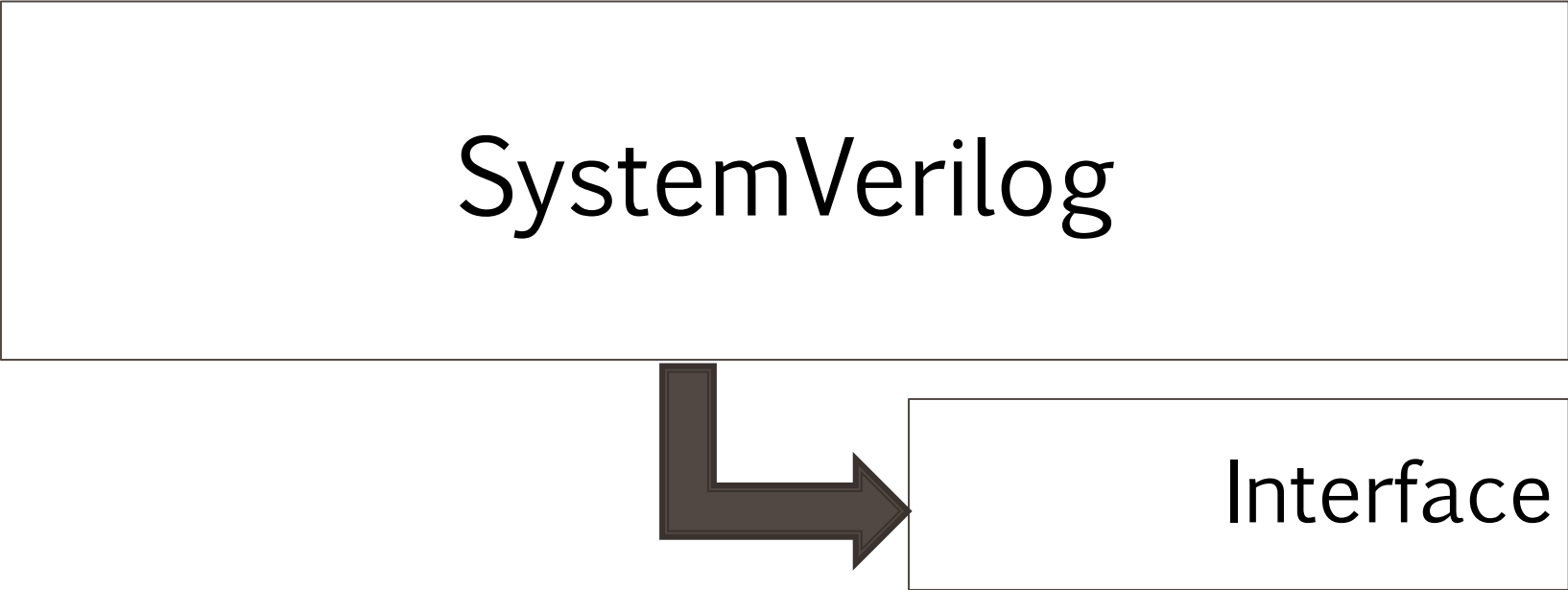
```
class Transaction extends BaseTr;  
    virtual function void display();  
        $display("In Transaction class");  
    endfunction  
endclass
```

```
module top;
```

```
    BaseTr btr;  
    Transaction tr;
```

```
    initial begin  
        //btr = new; // Error cannot be instantiated  
        tr = new;  
        tr.display();  
    end  
endmodule
```

```
ncsim> run  
In Transaction class
```

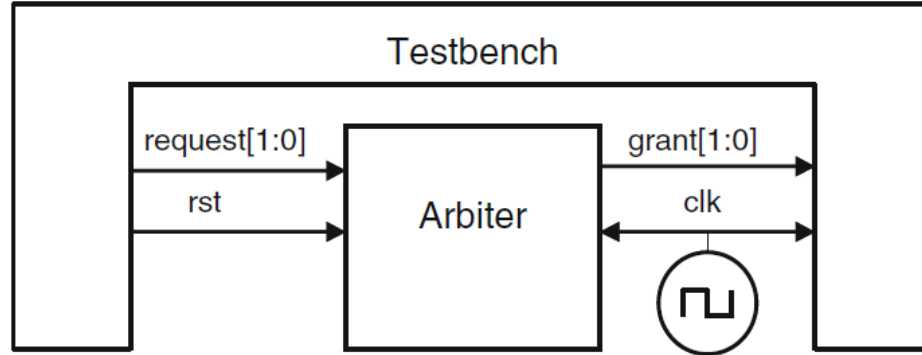


```
graph TD; A[SystemVerilog] --> B[Interface]
```

SystemVerilog

Interface

Traditional Connections



Without Interface (Traditional):

- Testbench connected to an arbiter, using individual signals

Challenges:

- Real designs with hundreds of pins require pages of signal and port declarations.
 - Error prone
 - For hierarchical designs repeated declarations and connections
 - Adding new signal need many changes

```
module arb_port (output logic [1:0] grant,  
    input logic [1:0] request,  
    input logic reset,  
    input logic clk);
```

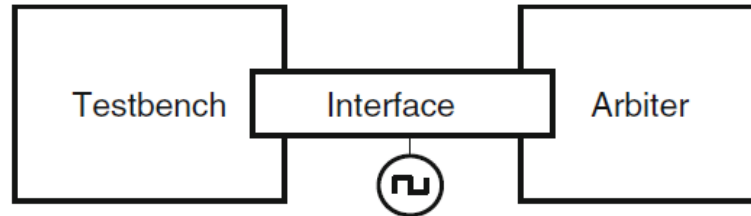
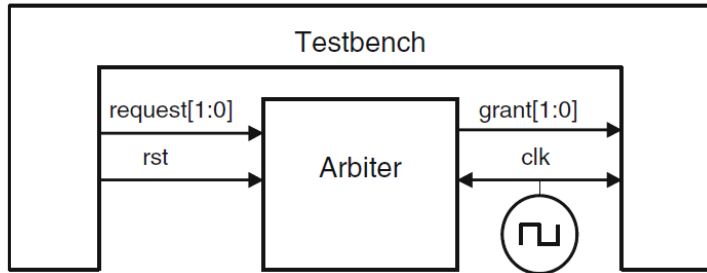
```
endmodule
```

```
module test (input logic [1:0] grant,  
    output logic [1:0] request,  
    output logic reset,  
    input logic clk);
```

```
endmodule
```

```
module top;  
    logic [1:0] grant, request;  
    logic clk=0, reset;  
    always #5 clk = ~clk;  
    arb_port a1 (grant, request, reset, clk);  
    test t1(grant, request, reset, clk);  
endmodule
```

Interface: Introduction



- Interface extends into the two blocks, representing the drivers and receivers
- Clock can be part of the interface or a separate port

```
interface arb_if(input bit clk);  
    logic [1:0] grant, request;  
    logic reset;  
endinterface
```

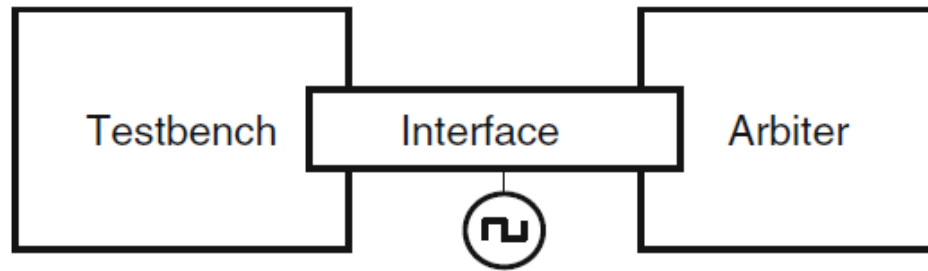
Interface:

- Bundle the wires together into an interface
- Can have variables, parameters, tasks, and functions (similar to a class)
- Can be instantiated similar to module
- Interface can have connectivity, synchronization, the functionality of the communication and error checking

Benefits:

- Enhances code reusability
- Reduces the chance of making wrong connections

Interface: Connection (1)



```
interface arb_if(input bit clk);  
    logic [1:0] grant, request;  
    logic reset;  
endinterface
```

- Interfaces are declared outside the module and program blocks

```
module test (arb_if arbif);  
  
endmodule
```

```
module arb (arb_if arbif);  
  
endmodule
```

```
module top;  
    bit clk;  
    always #5 clk = ~clk;  
  
    arb_if arbif(clk);  
  
    arb a1 (arbif);  
    test t1(arbif);  
endmodule
```


Interface: Connection (2)

- If legacy modules are existing (with ports and without interfaces), direct connection between signals of interfaces with ports can be made

```
module arb_port (output logic [1:0] grant,  
    input logic [1:0] request,  
    input logic reset,  
    input logic clk);
```

```
endmodule
```

```
module test (arb_if arbif);
```

```
endmodule
```

```
interface arb_if(input bit clk);  
    logic [1:0] grant, request;  
    logic reset;  
endinterface
```

```
module top;  
    bit clk;  
    always #5 clk = ~clk;
```

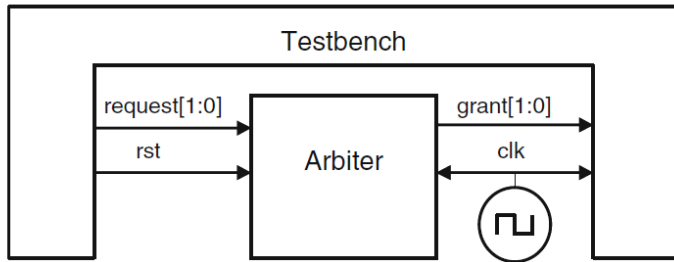
```
    arb_if arbif(clk);
```

```
    arb_port a1 (.grant (arbif.grant),  
                .request (arbif.request),  
                .reset (arbif.reset),  
                .clk (arbif.clk));
```

```
    test t1(arbif);
```

```
endmodule
```

Interface: **modport**



- Ports have direction and used by the tools for checking validity of connections
- Signals in the interfaces can have directions and grouped using **modport**
- Different views of interface

```
interface arb_if(input bit clk);
    logic [1:0] grant, request;
    logic reset;
endinterface
```

```
interface arb_if(input bit clk);
    logic [1:0] grant, request;
    logic reset;
```

```
    modport TEST (output request, reset, input grant, clk);
    modport DUT (input request, reset, clk, output grant);
    modport MONITOR (input request, grant, reset, clk);
endinterface
```

- A subset of signals can be chosen in **modport** (not shown in this example)

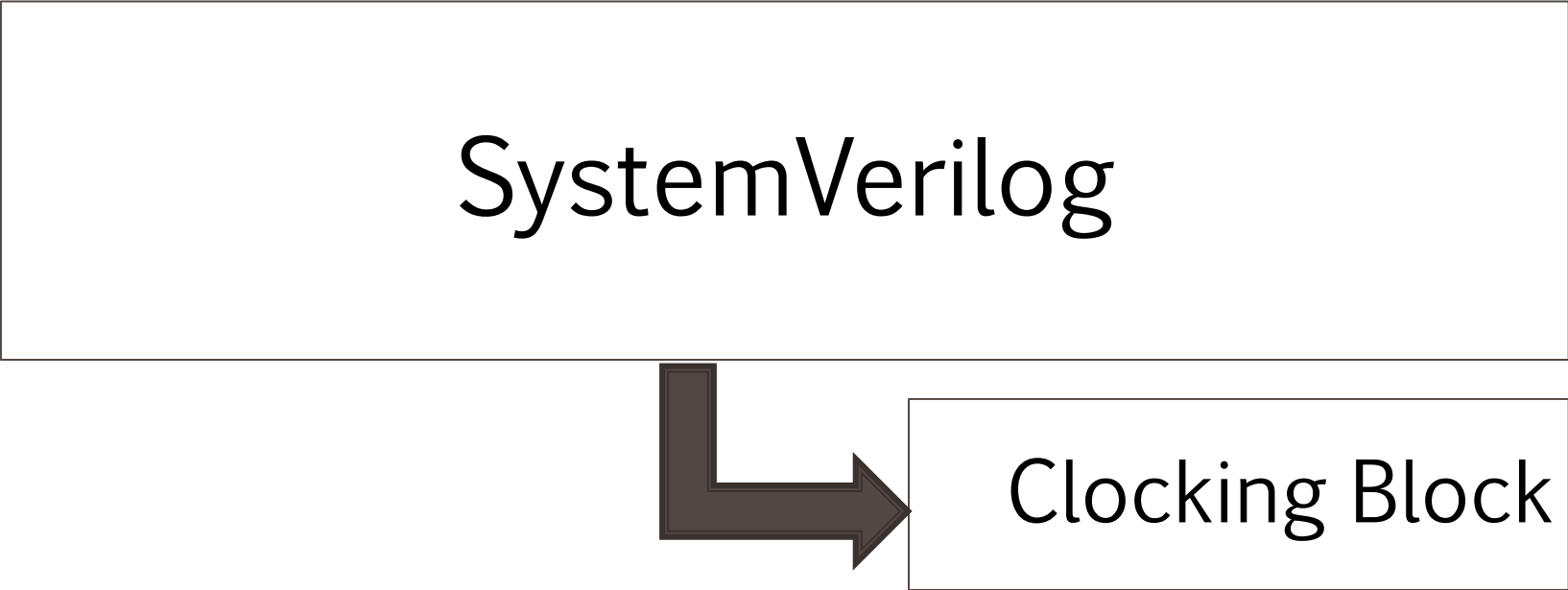
```
module arb_mod (arb_if.DUT arbif);
endmodule

module test_mod (arb_if.TEST arbif);
endmodule
```

```
module top;
    bit clk;
    always #5 clk = ~clk;

    arb_if arbif(clk);

    arb_mod a1 (arbif.DUT);
    test_mod t1(arbif.TEST);
endmodule
```



```
graph TD; A[SystemVerilog] --> B[Clocking Block]
```

SystemVerilog

Clocking Block

Clocking Block: Declaration

- Synchronous behavior of signals can be explicitly specified in the clocking block
 - Enclosed within **clocking** and **endclocking**
 - Can be specified in **program, module,** or **interface** [typically specified within **interface**]

Motivation

- Signals specified in a clocking block are driven (launched) and sampled (captured) synchronously with the given clock
- It ensures that testbench interacts with the signals at the right time

```
interface arb_if(input bit clk);  
    logic [1:0] grant, request;  
    logic reset;  
  
    clocking cb @(posedge clk); // Declare cb  
        output request;  
        input grant;  
    endclocking  
  
    modport TEST (clocking cb, // Use cb  
        output reset);  
  
    modport DUT (input request, reset, clk, output grant);  
  
endinterface
```

- Clocking block *cb* active on the positive edge of the *clk*
- Signals *request* and *grant* are driven and sampled by the clocking block (signal direction with respect to testbench)

Clocking Block: Usage

- Synchronous signals can be accessed using clocking block
 - Declaration of a clocking block is also treated as instantiation in the interface

```
module test_with_cb(arb_if.TEST arbif);
initial begin
    @arbif.cb;
    arbif.cb.request <= 2'b01;
    @arbif.cb;
    $display("@%0t: Grant = %b", $time, arbif.cb.grant);
    @arbif.cb;
    $display("@%0t: Grant = %b", $time, arbif.cb.grant);
    $finish;
end
endmodule : test
```

```
interface arb_if(input bit clk);
    logic [1:0] grant, request;
    logic reset;

    clocking cb @(posedge clk); // Declare cb
        output request;
        input grant;
    endclocking

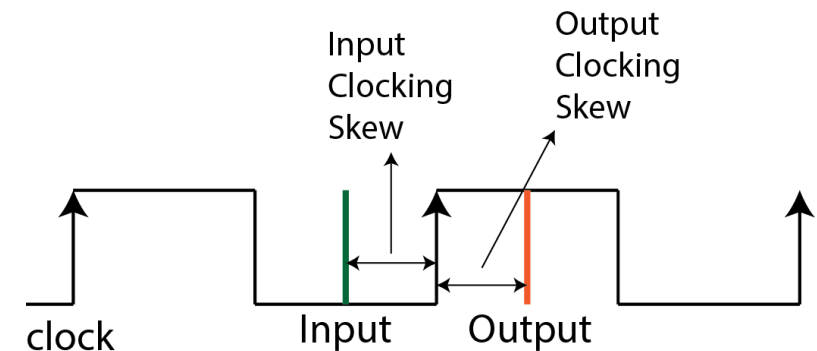
    modport TEST (clocking cb, // Use cb
        output reset);

    modport DUT (input request, reset, clk, output grant);
endinterface
```

Clocking Block: Timing

Clocking Skew

- Specified with respect to the clock edge
- Input Clocking skew specifies the moment (w.r.t clock edge) at which input is sampled
- Output Clocking skew specifies the moment the signal is driven



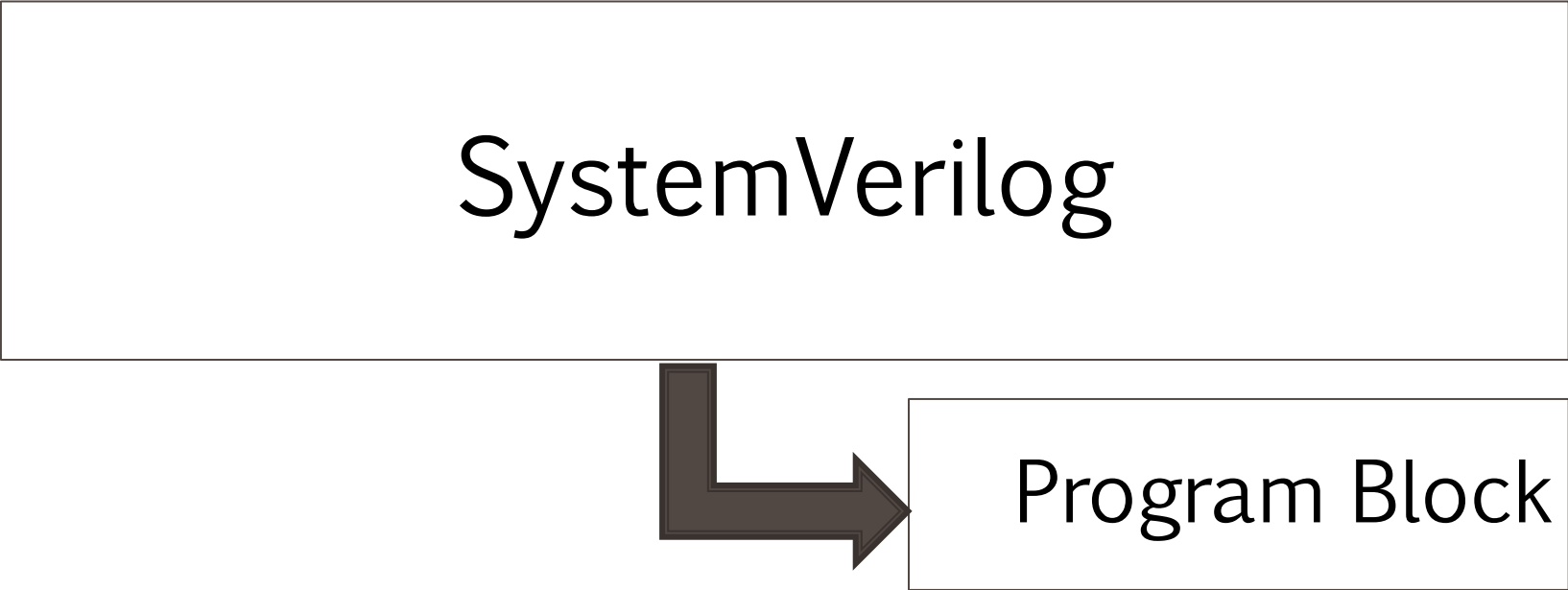
Timing (when we do not specify it in Clocking Block)

- Input signals are sampled just before the design executes (with a skew of #1step)
 - #1step is the time instant just before the clock changes
- Outputs are driven back into the design during the current time slot (with a delay of #0)

- Timing can be specified using **default** statement in the clocking block

```
clocking cb @(posedge clk); // Declare cb
    default input #15ns output #10ns;
    output request;
    input grant;
endclocking
```

- Inputs are sampled 15ns before the positive edge of the clock
- Outputs are driven 10ns after the positive edge of the clock



```
graph TD; A[SystemVerilog] --> B[Program Block]
```

SystemVerilog

Program Block

Program Block: Motivation

- There are many possibilities of the race conditions between the DUT and testbench in Verilog semantics
- The *memory* is waiting for positive edge of *start*
- In the *test*, when *start=1*, *write* is still 0
 - *memory* can enter the always block, while *write=0*
 - *addr* and *data* have new value, while *write* can have old value
- Testbench needs to:
 - Drive inputs of the DUT on or after the clock edge (all inputs should be seen updated by the DUT)
 - Read the outputs just before the following edge (as late as possible, after DUT has done the processing for all the outputs)

```
module memory(input wire start, write,
input wire [7:0] addr, data);
logic [7:0] mem[256];
always @(posedge start) begin
    if (write)
        mem[addr] = data;
    ...
end
endmodule

module test(output logic start, write,
output logic [7:0] addr, data);
initial begin
    start = 0; // Initialize signals
    write = 0;
    #10; // Short delay
    addr = 8'h42;
    data = 8'h5a;
    start = 1; // Start first command
    write = 1;
    ...
end
endmodule
```


Program Block: Example

- To separate timing behavior of a testbench and the design under test (DUT) **program block** is provided in SystemVerilog
 - Enclosed within **program**, and **endprogram**
 - Code for testbench normally written in the program block

Race conditions are avoided between design and testbench as follows:

- All elements within the program block are scheduled to execute after active region and NBA update (in the **reactive region**)
- All signals are sampled after DUT has done computation (in the **postponed region**)

```
interface arb_if(input bit clk);
    logic [1:0] grant, request;
    logic reset;

    clocking cb @(posedge clk); // Declare cb
        output request;
        input grant;
    endclocking

    modport TEST (clocking cb, // Use cb
        output reset);

    modport DUT (input request, reset, clk, output grant);

endinterface

program automatic test(arb_if.TEST arbif);
initial begin
    @arbif.cb;
    arbif.cb.request <= 2'b01;
    $display("@%0t: Drove req = 01", $time);
    $display("@%0t: Grant Read 1 = %b", $time, arbif.cb.grant);
    @arbif.cb;
    $display("@%0t: Grant Read 2 = %b", $time, arbif.cb.grant);
    @arbif.cb;
    $display("@%0t: Grant Read 3 = %b", $time, arbif.cb.grant);
    @arbif.cb;
    $display("@%0t: Grant Read 4 = %b", $time, arbif.cb.grant);
end
endprogram : test
```

SystemVerilog: Timing Regions

SystemVerilog introduces new timing regions in the Stratified Event Queue

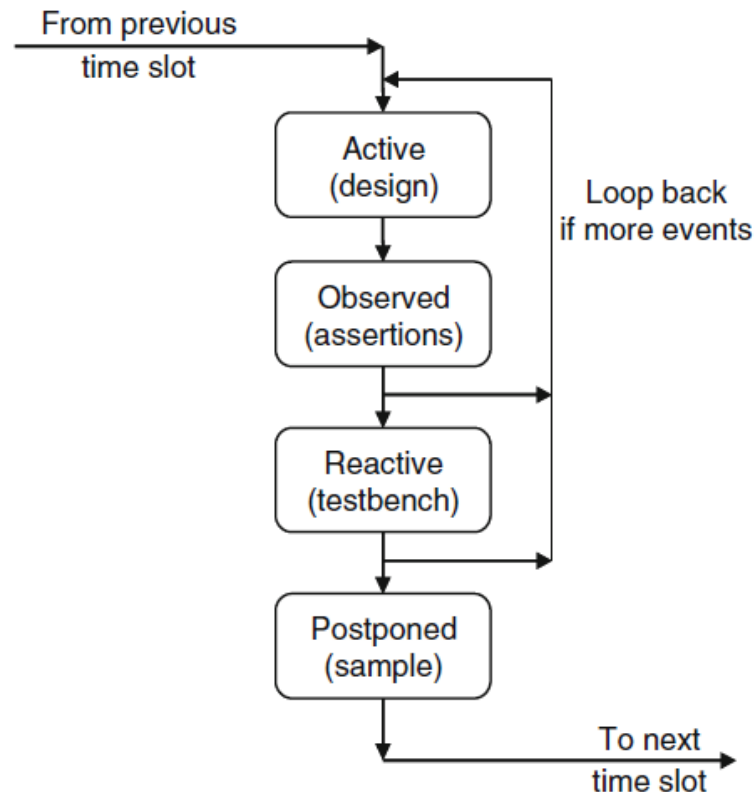


Table 4.1 Primary SystemVerilog scheduling regions

<i>Name</i>	<i>Activity</i>
Active	Simulation of design code in modules
Observed	Evaluation of SystemVerilog Assertions
Reactive	Execution of testbench code in programs
Postponed	Sampling design signals for testbench input

1. Active region, where design events run (RTL, gate-level and clock generator): **Active**, **Inactive** and **NBA update** of Verilog 2005 stratified Queue executed (recall VDF)
2. Observed region: SystemVerilog Assertions evaluated.
3. Reactive region: Testbench code in a program executes.
 - Events in the Observed and Reactive regions can trigger further design events in the Active region in the current cycle.
4. Postponed region: samples signals at the end of the time slot, in the read-only period, after design activity has completed (\$monitor, \$strobe part of it)

References

- Spear, Chris. SystemVerilog for Verification: A Guide to Learning the Testbench Language Features. Vol. 161. Springer, 2008.